

✓ Homework 2: Programming

The following notebook contains skeleton-code for answering problems 2 and 3 of homework assignment 2. Please read through each cell carefully to understand what is expected to be implemented. For your final submission, please try to clean up any intermediate outputs used for debugging.

✓ Imports

You should be able to complete the entire assignment using only the following imports. Please consult the course staff if you are unsure about whether additional packages may be used.

```
## Import Packages
import random
import numpy as np
import matplotlib.pyplot as plt
```

✓ Question 2



Below we provide an AutoGrad class named `Value`. The basic idea is to store the existing computational map during the creation of each `Value`, and calculate the gradient using backpropagation when one of the `Value` calls `backward()` method.

The `backward()` function will arrange the computational graph and backpropagate the gradients. All you need to do is to implement all the operations with its corresponding `_backward` function. We have provided the `__add__` function (sum of two nodes) as an example to help get you started.

This notebook is designed in a Object Oriented way, if you are not familiar with the Object Oriented Programming in Python, you can refer to:

- (1) <https://realpython.com/python3-object-oriented-programming/>
- (2) <https://docs.python.org/3/tutorial/classes.html>

```
class Value:

    """
    Basic unit of storing a single scalar value and its gradient
    """

    def __init__(self, data, _children=()):
        """

        self.data = data
        self.grad = 0
        self._prev = set(_children)
        self._backward = lambda: None

    def __add__(self, other):
        """
        Example implementation of a single class operation (addition)

        Args:
            other (Any): Node to add with the class

        Returns:
            out (callable): Function to referesh the gradient
        """
        #Firstly, convert some default value type in python to Value
```

```

#Then do operations with two or more Value object
other = other if isinstance(other, Value) else Value(other)

#Secondly, create a new Value object which is the result of the
out = Value(self.data + other.data, (self, other))

#Thirdly, create a _backward function for the output object to r
# the gradient of its _childrens,
#Then assign this _backward function to the output object.
def _backward():
    self.grad += out.grad * 1.0
    other.grad += out.grad * 1.0
out._backward = _backward

return out

def __mul__(self, other):
    """
    Multiplication operation (e.g. Value(3) * Value(2) = Value(6))
    """
    #TODO implement multiplication operation

    #Firstly, convert some default value type in python to Value
    #Then do operations with two or more Value object
    other = other if isinstance(other, Value) else Value(other)

    #Secondly, create a new Value object which is the result of the
    out = Value(self.data * other.data, (self, other))

    def _backward():
        self.grad += out.grad * other.data
        other.grad += out.grad * self.data
    out._backward = _backward

    return out

def __pow__(self, other):
    """
    Power operation (e.g Value(3) ** 2 = Value(9))
    """
    #TODO implement power operation, we don't need to convert the ex
    assert isinstance(other, (int, float))

    out = Value(self.data ** other, (self,))

```

```
def _backward():
    self.grad += other * (self.data ** (other - 1)) * out.grad
    out._backward = _backward

    return out

def relu(self):
    """
    ReLU activation function applied to the current Value
    """
    #TODO implement the relu activation function for the value itself

    out = Value(self.data if self.data > 0 else 0, (self,))

    def _backward():
        self.grad += (1.0 if self.data > 0 else 0.0) * out.grad
        out._backward = _backward

    return out

def exp(self):
    """
    Exponentiate the current Value (e.g.  $e^{\text{Value}(0)} = \text{Value}(1)$ )
    """
    #TODO implement the exponential function for and treat the value
    #The base is natural e, you can use numpy to calculate the value

    out = Value(np.exp(self.data), (self,))

    def _backward():
        self.grad += out.data * out.grad
        out._backward = _backward

    return out

def log(self):
    """
    Take the natural logarithm (base e) of the current Value
    """
    #TODO implement the logarithm function for and treat the value as
    #The bottom number should be e, you can use numpy to calculate it

    out = Value(np.log(self.data), (self,))
```

```

def _backward():
    self.grad += (1 / self.data) * out.grad
    out._backward = _backward

    return out

def backward(self):
    """
    Run backpropagation from the current Value
    """
    #This function is called when you start backpropagation from this node
    #The gradient of this value is initialized to 1 for you.
    self.grad = 1

    #You need to find a right topological order all of the children
    #As for topology sort, you can refer to http://www.cs.cornell.edu

    topo = []
    #TODO find the right list of Value to be traversed
    ...

    Hint: you can recursively visit all non-visited node from the node with no children
    add one node to the head of the list after all of its children are visited
    ...

    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)

    # Build the topological order
    build_topo(self)

    #go one variable at a time and apply the chain rule to get its gradient
    for v in reversed(topo):
        v._backward()

    # We handled the negation and reverse operations for you
    def __neg__(self): # -self
        """
        Negate the current Value

```

```

    """
    return self * -1

def __radd__(self, other): #other + self
    """
    Reverse addition operation (ordering matters in Python)
    """
    return self + other

def __sub__(self, other): # self - other
    """
    Subtraction operation
    """
    return self + (-other)

def __rsub__(self, other): # other - self
    """
    Reverse subtraction operation
    """
    return other + (-self)

def __rmul__(self, other): # other * self
    """
    Reverse multiplication operation
    """
    return self * other

def __truediv__(self, other): # self / other
    """
    Division operation
    """
    return self * other**-1

def __rtruediv__(self, other): # other / self
    """
    Reverse diction operation
    """
    return other * self**-1

def __repr__(self):
    """
    Class representation (instead of unfriendly memory address)
    """
    return f"Value(data={self.data}, grad={self.grad})"
```

Now, we are going to use the simple example in q1.b to get you familiar with the usage of this class.

If your implementation is correct, you will get the same values and gradients as your hand-caculated ones.

Be careful! Even you get this test case right, it does not guarantee the correctness of your implementation.

```
## Initialize Example Values (From Written Assignment)
# w1 = Value(0.3)
# w2 = Value(-0.5)
# x1 = Value(0.2)
# x2 = Value(0.4)

w1 = Value(0.2)
w2 = Value(0.4)
x1 = Value(-0.4)
x2 = Value(0.5)

#TODO
#Do calculation for the question 1.b, and call backward to start backpro
#Then print out the gradient of w1 w2 x1 x2.
z = w1 * x1 + w2 * x2
sigmoid = (1 + (-z).exp()) ** -1
reg = 0.5 * (w1**2 + w2**2)
result = sigmoid + reg

result.backward()
print("w1 grad =", w1.grad)
print("w2 grad =", w2.grad)
print("x1 grad =", x1.grad)
print("x1 grad =", x2.grad)
```

```
w1 grad = 0.10035913775925998
w2 grad = 0.524551077800925
x1 grad = 0.04982043112037002
x1 grad = 0.09964086224074004
```

```
def track_computation(w1, w2, x1, x2):
    print("Forward Pass:")
```

```
# Step 1:  $w_1 * x_1$ 
w1x1 = w1 * x1
print(f"w1 * x1 = {w1x1.data:.4f}")

# Step 2:  $w_2 * x_2$ 
w2x2 = w2 * x2
print(f"w2 * x2 = {w2x2.data:.4f}")

# Step 3:  $w_1x_1 + w_2x_2$ 
z = w1x1 + w2x2
print(f"w1x1 + w2x2 = {z.data:.4f}")

# Step 4:  $-z$ 
neg_z = -z
print(f"-z = {neg_z.data:.4f}")

# Step 5:  $\exp(-z)$ 
exp_neg_z = neg_z.exp()
print(f"exp(-z) = {exp_neg_z.data:.4f}")

# Step 6:  $1 + \exp(-z)$ 
one_plus_exp = Value(1) + exp_neg_z
print(f"1 + exp(-z) = {one_plus_exp.data:.4f}")

# Step 7:  $(1 + \exp(-z)) ** -1$  (sigmoid)
sigmoid = one_plus_exp ** -1
print(f"sigmoid = {sigmoid.data:.4f}")

# Step 8:  $w_1^2$ 
w1_sq = w1 ** 2
print(f"w1^2 = {w1_sq.data:.4f}")

# Step 9:  $w_2^2$ 
w2_sq = w2 ** 2
print(f"w2^2 = {w2_sq.data:.4f}")

# Step 10:  $w_1^2 + w_2^2$ 
w_sq_sum = w1_sq + w2_sq
print(f"w1^2 + w2^2 = {w_sq_sum.data:.4f}")

# Step 11:  $0.5 * (w_1^2 + w_2^2)$ 
reg = Value(0.5) * w_sq_sum
print(f"regularization = {reg.data:.4f}")
```

```

# Step 12: sigmoid + regularization
result = sigmoid + reg
print(f"result = {result.data:.4f}")

return result, [w1, w2, x1, x2, w1*x1, w2*x2, z, neg_z, exp_neg_z,
                one_plus_exp, sigmoid, w1_sq, w2_sq, w_sq_sum, reg, r

# Helper function to print gradients
def print_gradients(variables, labels):
    print("\nBackward Pass:")
    for var, label in zip(variables, labels):
        print(f"grad {label} = {var.grad:.4f}")

# Main computation
def run():
    # Initialize values
    w1 = Value(0.2)
    w2 = Value(0.4)
    x1 = Value(-0.4)
    x2 = Value(0.5)

    # Track computation and get all intermediate variables
    result, all_vars = track_computation(w1, w2, x1, x2)

    # Perform backpropagation
    result.backward()

    # Print gradients
    labels = ["w1", "w2", "x1", "x2", "w1*x1", "w2*x2", "z", "-z", "exp(
              "1+exp(-z)", "sigmoid", "w1^2", "w2^2", "w1^2+w2^2", "regu

    print_gradients(all_vars, labels)

run()

```

```

Forward Pass:
w1 * x1 = -0.0800
w2 * x2 = 0.2000
w1*x1 + w2*x2 = 0.1200
-z = -0.1200
exp(-z) = 0.8869
1 + exp(-z) = 1.8869
sigmoid = 0.5300
w1^2 = 0.0400

```

```

w2^2 = 0.1600
w1^2 + w2^2 = 0.2000
regularization = 0.1000
result = 0.6300

Backward Pass:
grad w1 = 0.1004
grad w2 = 0.5246
grad x1 = 0.0498
grad x2 = 0.0996
grad w1*x1 = 0.2491
grad w2*x2 = 0.2491
grad z = 0.2491
grad -z = -0.2491
grad exp(-z) = -0.2809
grad 1+exp(-z) = -0.2809
grad sigmoid = 1.0000
grad w1^2 = 0.5000
grad w2^2 = 0.5000
grad w1^2+w2^2 = 0.5000
grad regularization = 1.0000
grad result = 1.0000

```

✓ Question 3

✓ Implementation of the linear layer

You will implement a `LinearLayer` module here.

We provide the initialization of the class `LinearLayer`. You need to implement the forward function -- Return the results - `out` with the shape `[n_samples, n_out_channels]` of a linear layer when the data `x` shaped `[n_samples, n_in_channels]` is fed into it.

```

class Module:

    """
    Base Model Module
    """

    def parameters(self):
        """

```

```

        """
        return []

def zero_grad(self):
    """

    """
    for p in self.parameters():
        p.grad = 0

class LinearLayer(Module):
    """
    Linear Layer
    """

    def __init__(self, nin, nout):
        """
        Here we randomly initilize the weights w as 2-dimensional list o
        And b as 1-dimensional list of Values with value 0

        You may use this stucture to implement the __call__ function
        """
        self.w = []
        for i in range(nin):
            w_tmp = [Value(random.uniform(-1,1)) for j in range(nout)]
            self.w.append(w_tmp)
        self.b = [Value(0) for i in range(nout)]
        self.nin = nin
        self.nout = nout

    def __call__(self, x):
        """
        Args:
            x (2d-list): Two dimensional list of Values with shape [batch_size, nin]

        Returns:
            xout (2d-list): Two dimensional list of Values with shape [batch_size, nout]
        """
        #TODO implement this function and return the output of a linear
        batch_size = len(x)
        out = [[Value(0) for _ in range(self.nout)] for _ in range(batch_size)]

        for i in range(batch_size):

```

```
        for j in range(self.nout):
            total = Value(0)
            for k in range(self.nin):
                total = total + self.w[k][j] * x[i][k]
            total = total + self.b[j]
            out[i][j] = total

    return out

def parameters(self):
    """
    Get the list of parameters in the Linear Layer

    Args:
        None

    Returns:
        params (list): List of parameters in the layer
    """
    return [p for row in self.w for p in row] + [p for p in self.b]
```

Test your implementation of linear layer, the error should be nearly 0.

```
## Initialization of Layer with Weights
linear_model_test = LinearLayer(4, 4)
linear_model_test.w = [[Value(data=0.7433570245252463), Value(data=-0.96
                        [Value(data=-0.1414882837892344), Value(data=-0.5
                        [Value(data=0.3990701306597799), Value(data=-0.33
                        [Value(data=-0.5711035064293541), Value(data=-0.0
linear_model_test.b = [Value(data=0), Value(data=0), Value(data=0), Valu

## Forward Pass
x_test = [[-0.17120438454836173, -0.3736077734087335, -0.484954130546532
y_hat_test = linear_model_test(x_test)
y_ref = [[Value(data=-0.7401928625441141), Value(data=0.5466095223360173

## Error Calculation
predict_error = 0
for i in range(4):
    predict_error += (y_hat_test[0][i] - y_ref[0][i])**2
print(predict_error.data)
```

0.0

✓ Implementation of Loss functions

You will implement softmax, cross entropy loss, and accuracy here for further use

```
def softmax(y_hat):  
    """  
    Softmax computation  
  
    Args:  
        y_hat (2d-list): 2-dimensional list of Values with shape [batch_  
  
    Returns:  
        s (2d-list): 2-dimensional list of Values with the same shape as  
    """  
    #TODO implement the softmax function and return the output.  
    batch_size = len(y_hat)  
    n_class = len(y_hat[0])  
    sums = [Value(0) for _ in range(batch_size)]  
    s = [[Value(0) for _ in range(n_class)] for _ in range(batch_size)]  
  
    for i in range(batch_size):  
        for j in range(n_class):  
            s[i][j] = y_hat[i][j].exp()  
            sums[i] = sums[i] + s[i][j]  
  
    for i in range(batch_size):  
        for j in range(n_class):  
            s[i][j] = s[i][j] / sums[i]  
  
    return s
```

```
def cross_entropy_loss(y_hat, y):
    """
    Cross-entropy Loss computation

    Args:
        y_hat (2d-list): Output from linear function with shape [batch_size, num_classes]
        y (1d-list): List of ground truth labels with shape [batch_size,]

    Returns:
        loss (Value): Loss value of type Value
    """
    #TODO implement the calculation of cross_entropy_loss between y_hat and y
    s = softmax(y_hat)
    batch_size = len(y_hat)
    loss = Value(0)
    for i in range(batch_size):
        loss = loss + -s[i][y[i]].log() # all other terms sum to zero
    return loss / batch_size
```

```
def accuracy(y_hat, y):
    """
    Accuracy computation

    Args:
        y_hat (2d-list): Output from linear function with shape [batch_size, num_classes]
        y (1d-list): List of ground truth labels with shape [batch_size,]

    Returns:
        acc (float): Accuracy score
    """
    prediction_scores = np.array([[val.data for val in row] for row in y_hat])
    predicted_class = np.argmax(prediction_scores, axis=1)
    acc = np.sum(predicted_class == np.array(y)) / len(y)

    return acc
```

Test the implementation of `softmax()` and `cross_entropy_loss()` as well as the gradient calculation of `Value` class. The errors should be nearly 0.

```

## Ground Truth + Forward Pass
y_gt = [1]
y_hat_test = linear_model_test(x_test)

## Softmax Calculation
prob_test = softmax(y_hat_test)
prob_ref = [[0.10441739448437284, 0.37811510516540814, 0.416642899167655
softmax_error = 0
for i in range(4):
    softmax_error += (prob_ref[0][i] - prob_test[0][i])**2
print(softmax_error.data)

## Cross Entropy Loss Calculation
loss_test = cross_entropy_loss(y_hat_test, y_gt)
loss_ref = Value(data=0.9725566186970217)
print((loss_test - loss_ref).data)

## Update Gradient Based on Loss
linear_model_test.zero_grad()
loss_test.backward()
w_gradient_ref = [[-0.017876715758840547, 0.10646942068007896, -0.071331
                  [-0.0390111502584479, 0.23234103087567629, -0.15566102
                  [-0.05063764675610328, 0.30158564847453107, -0.2020526
                  [0.08634490197366762, -0.5142494748940867, 0.344530625
b_gradient_ref = [0.10441739448437282, -0.6218848948345919, 0.4166428991

## Compute Error
w_gradient_error = 0
b_gradient_error = 0
for i in range(4):
    b_gradient_error += (linear_model_test.b[i].grad - b_gradient_ref[i]
    for j in range(4):
        w_gradient_error += (linear_model_test.w[i][j].grad - w_gradient
print(w_gradient_error)
print(b_gradient_error)

```

```

0.0
0.0
0.0
0.0

```

Implement the following functions to visualize the ground truth and the decision boundary in the same figure.

```
def plot_points(X, Y, scale, n, data):
    """
    Plot points in the visualization image
    """
    points_color = [[0., 0. , 255.], [255., 0., 0.], [0., 255., 0.], [0., 0., 255.]]

    for i in range(X.shape[0]):
        #TODO Assign a color to "data" according to the position and the label
        x1, x2 = X[i]
        pixel_i = int((x1 + scale) * (n-1) / (2 * scale))
        pixel_j = int((x2 + scale) * (n-1) / (2 * scale))
        color = points_color[int(Y[i])]
        data[pixel_i][pixel_j] = color

    return data

def plot_background(scale, n, model):
    """
    Color the background in the visualization image
    """

    background_color = [[0., 191., 255.], [255., 110., 180.], [202., 255., 180.]]

    data = np.zeros((n,n,3), dtype='uint8')

    for i in range(n):
        x1 = -scale + 2 * scale / n * i
        for j in range(n):
            x2 = -scale + 2 * scale / n * j
            input = [[Value(x1),Value(x2)]]
            #TODO using the model to predict a class for the input and assign the color
            y_hat = model(input)
            predictions = [val.data for val in y_hat[0]]
            pred_idx = np.argmax(predictions)
            data[i][j] = background_color[pred_idx]

    return data

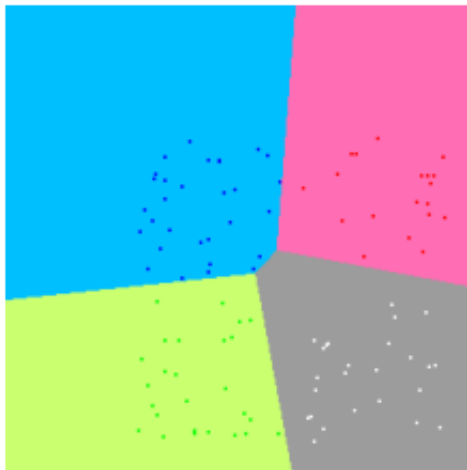
def visualization(X, Y, model):
```

```
def visualization(X, Y, model):
    """
    Decision boundary visualization
    """
    scale = 4.5 # the scale of X axis and Y axis. To say, x is from -scale to scale
    n = 300     # separate the image into n*n pixels

    data = plot_background (scale, n, model)
    data = plot_points (X, Y, scale, n, data)

    plt.imshow(data)
    plt.axis('off')
    plt.show()
```

if you implement the plot function correctly, you will get some image like:



✓ Implementation of training procedure

With input data `x`, ground_truth `y`, and `model` as parameters, implement the gradient descent method to train your model and plot loss and accuracy vs training iterations

```
def train(x,
          y,
          model,
          loss_function=cross_entropy_loss,
          accuracy_function=accuracy,
          max_iteration=500,
          learning_rate=1):
    """
```

Args:

x (2-d list): List of Values with shape: [n_samples, n_channels]
 y (1-d list): List of integers with shape: [n_samples]
 model (Module): Linear model
 loss_function (callable): Loss function to use during training
 accuracy_function (callable): Function used for calculating train
 max_iteration (int): Number of epochs to train model for
 learning_rate (numeric): Step size of the gradient update

"""

```
for i in range(max_iteration):
    #TODO compute y_hat and calculate the loss between y_hat and y a
    # the accuracy of y_hat w.r.t y.
    y_hat = model(x)
    loss = loss_function(y_hat, y)
    acc = accuracy_function(y_hat, y)

    #TODO Then You will need to calculate gradient for all parameter
    #do gradient descent for all the parameters.
    #The list of parameters can be easily obtained by calling
    #model.parameters() which is implemented above.
    model.zero_grad()
    loss.backward()

    for param in model.parameters():
        param.data -= learning_rate * param.grad

    #Then plot the loss / accuracy vs iterations.
    if i % 20 == 19:
        print("iteration",i,"loss:",loss.data, "accuracy:",acc)
    ## record loss
    if i == 0 :
        # initialize L
        L = loss.data
        A = acc
    else:
        L = np.append(L,loss.data)
        A = np.append(A,acc)

    ## Plot Loss and Accuracy
    fig0=plt.figure(0)
    plt.plot(L,'-')
    plt.xlabel('Iteration', fontsize=18)
    plt.ylabel('Loss', fontsize=16)
    plt.show()
    fig1=plt.figure(1)
```

```
plt.plot(A, '-')
plt.xlabel('Iteration', fontsize=18)
plt.ylabel('Accuracy', fontsize=16)
plt.show()
```

✓ Train the model

Load the data, format it, instantiate your model and start training!

```
## Load Q3 Dataset
datapath = './HW2_Q3_dataset/Q3_data.npz'
data = np.load(datapath)

## Load Data and Parse Shape Information
X = data['X']
Y = data['Y']
print(X.shape, Y.shape, np.unique(Y))
nin = X.shape[1]
nout = np.max(Y) + 1

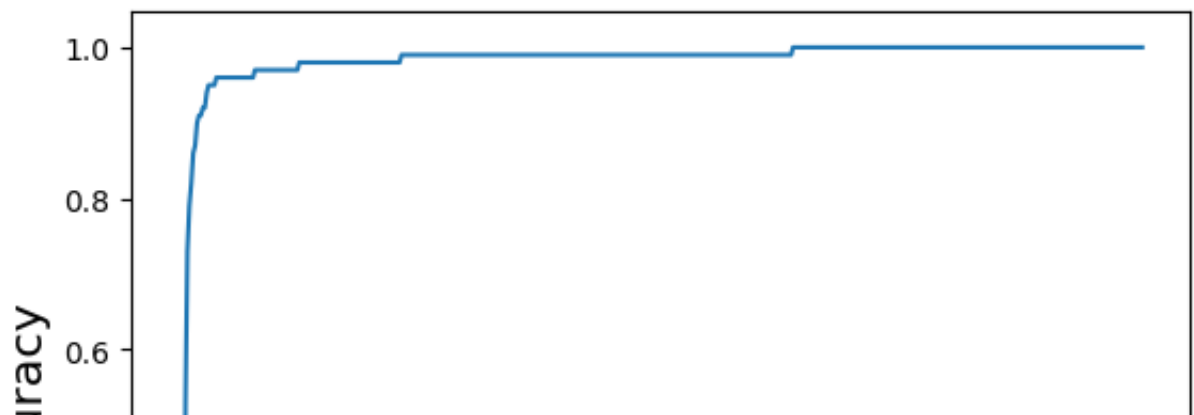
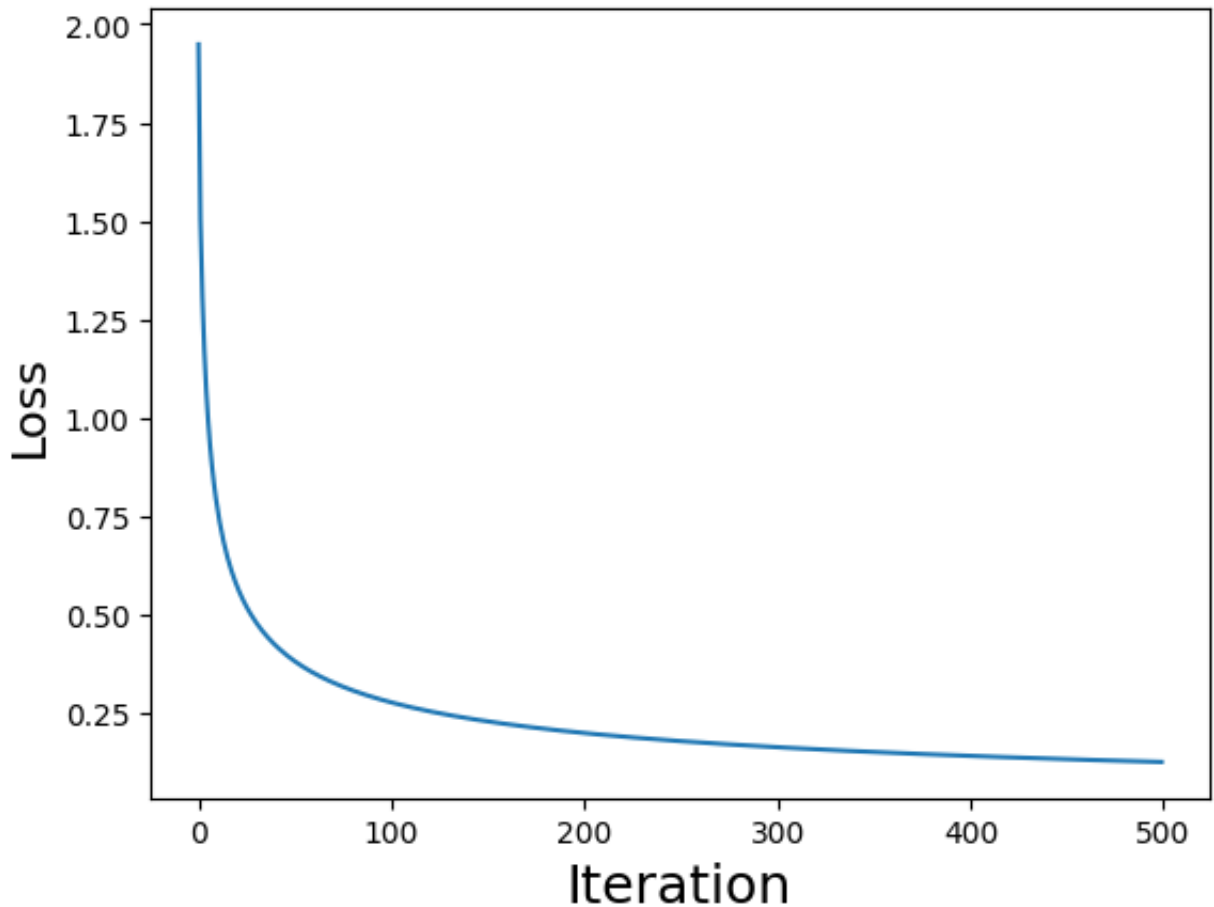
## Initialize data using your Value class
x = [[Value(v) for v in sample] for sample in X]
y = [int(v) for v in Y]

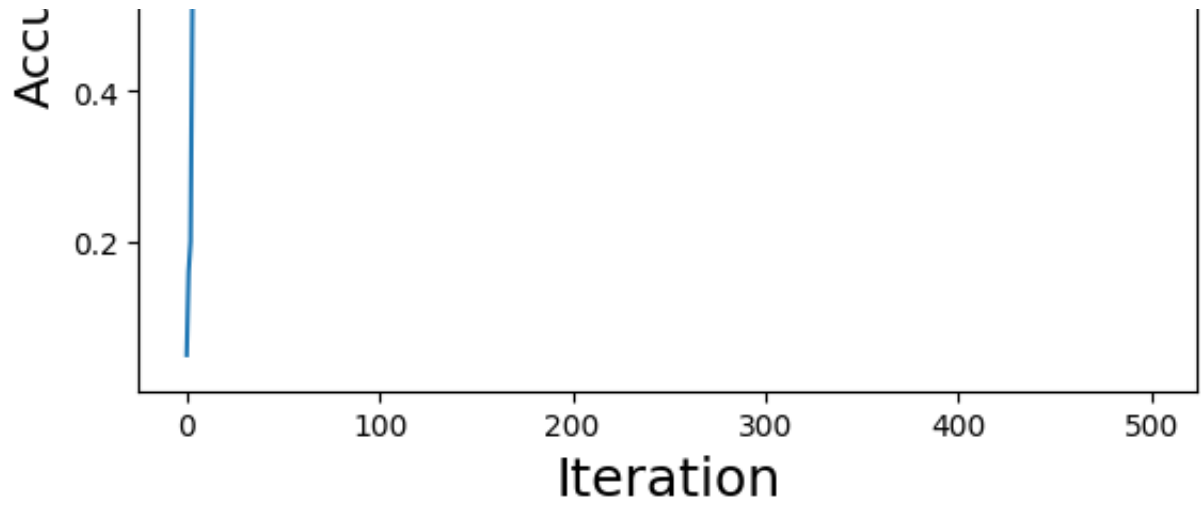
## Initialize a Linear Model
linear_model = LinearLayer(nin, nout)

## Train the Model using Your Data
train(x, y, linear_model)
```

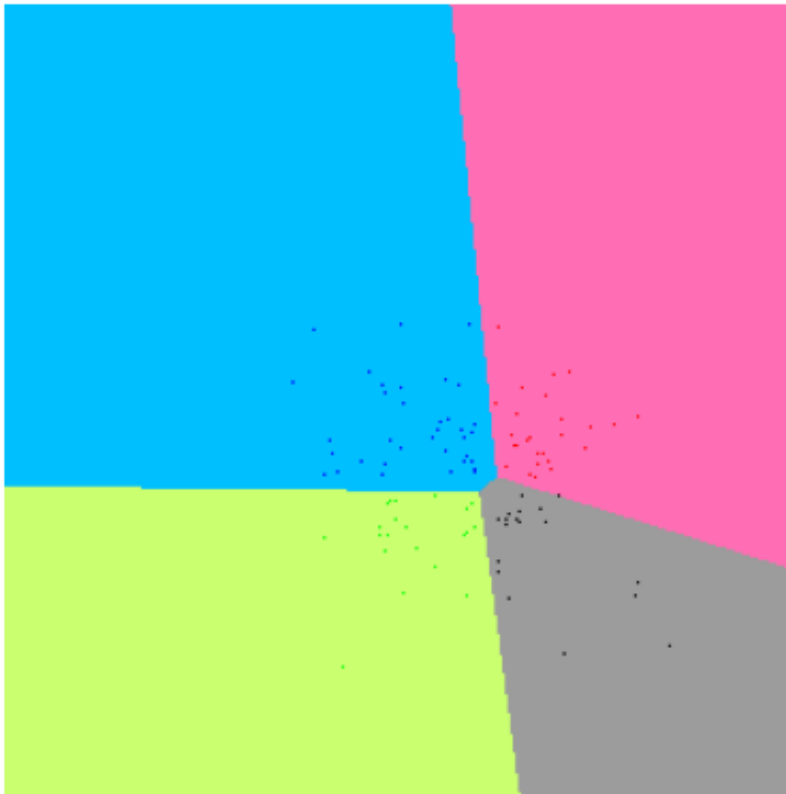
```
(100, 2) (100,) [0 1 2 3]
iteration 19 loss: 0.5824707008301163 accuracy: 0.96
iteration 39 loss: 0.42431192622709113 accuracy: 0.97
iteration 59 loss: 0.3514223531324079 accuracy: 0.97
iteration 79 loss: 0.30694694886459145 accuracy: 0.98
iteration 99 loss: 0.2760957226736333 accuracy: 0.98
iteration 119 loss: 0.25303401419632093 accuracy: 0.99
iteration 139 loss: 0.2349234863211334 accuracy: 0.99
iteration 159 loss: 0.22019296320476142 accuracy: 0.99
iteration 179 loss: 0.20789221856125772 accuracy: 0.99
iteration 199 loss: 0.1974081054246296 accuracy: 0.99
iteration 219 loss: 0.18832484657752113 accuracy: 0.99
iteration 239 loss: 0.1803491872846508 accuracy: 0.99
```

```
iteration 259 loss: 0.1732675612768003 accuracy: 0.99
iteration 279 loss: 0.16692024470599884 accuracy: 0.99
iteration 299 loss: 0.16118506748806852 accuracy: 0.99
iteration 319 loss: 0.15596676731298761 accuracy: 1.0
iteration 339 loss: 0.15118981288497685 accuracy: 1.0
iteration 359 loss: 0.14679343515374627 accuracy: 1.0
iteration 379 loss: 0.1427281064217244 accuracy: 1.0
iteration 399 loss: 0.1389529939648655 accuracy: 1.0
iteration 419 loss: 0.13543408480414482 accuracy: 1.0
iteration 439 loss: 0.13214278222692877 accuracy: 1.0
iteration 459 loss: 0.129054840005711 accuracy: 1.0
iteration 479 loss: 0.12614954235664536 accuracy: 1.0
iteration 499 loss: 0.12340906539876766 accuracy: 1.0
```





```
## Visualize learned decision boundaries  
visualization(X, Y, linear_model)
```



✓ Question 4

✓ a) Is this dataset linear separable?

load the dataset for this question and train a linear model on this dataset and report the performance

```
## Load Q4 Dataset
datapath = './HW2_Q4_dataset/Q4_data.npz'
data = np.load(datapath)

## Parse Data and Identify Dimensions
X = data['X']
Y = data['Y']
nin = X.shape[1]
nout = int(np.max(Y)) + 1

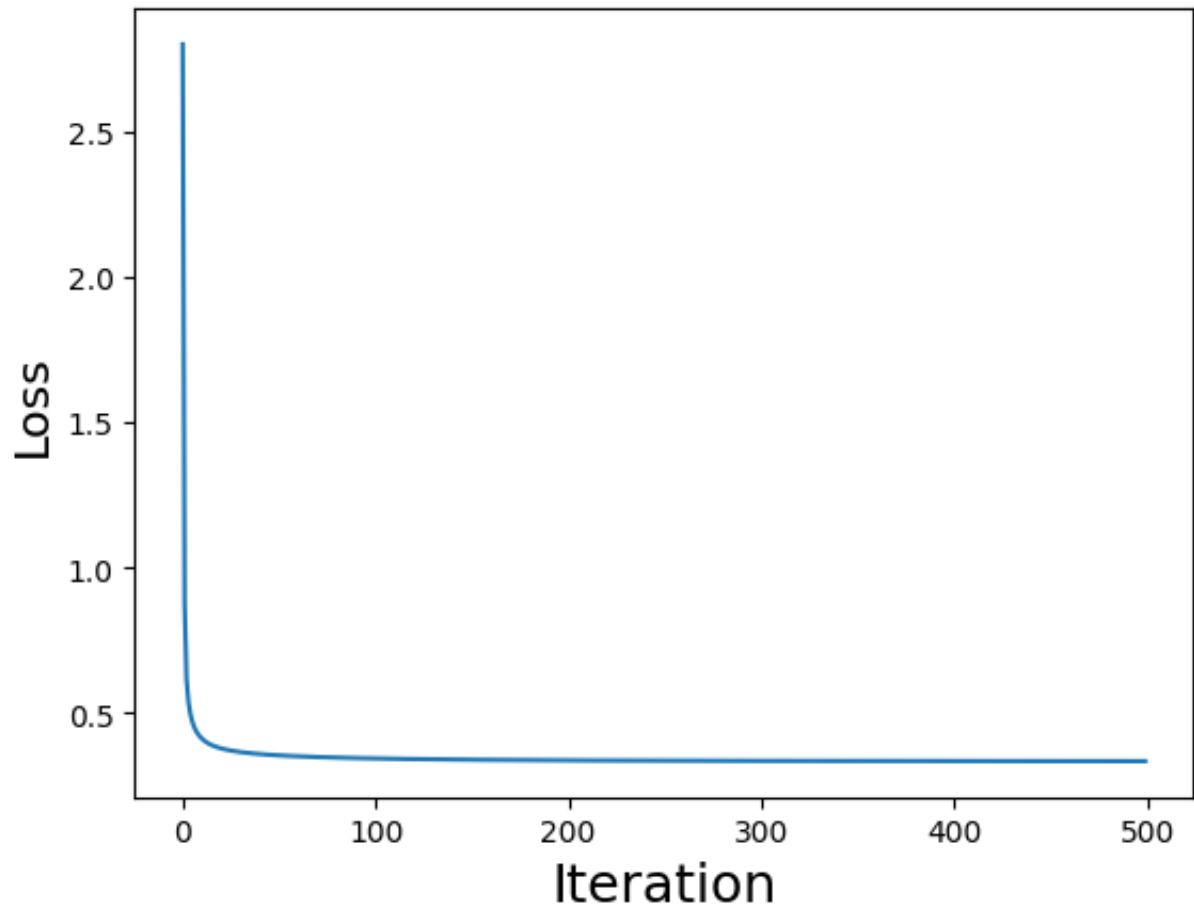
## Initialize data using your value class
x = [[Value(v) for v in sample] for sample in X]
y = [int(v) for v in Y]
```

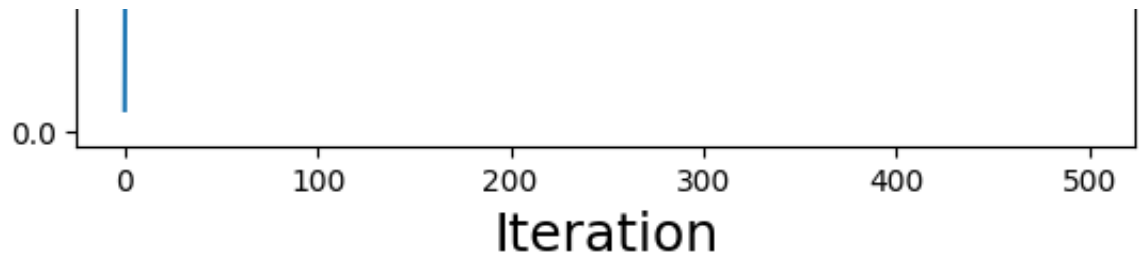
```
## Initialize Linear Model
linear_model = LinearLayer(nin, nout)

## Train Model
train(x, y, linear_model)
```

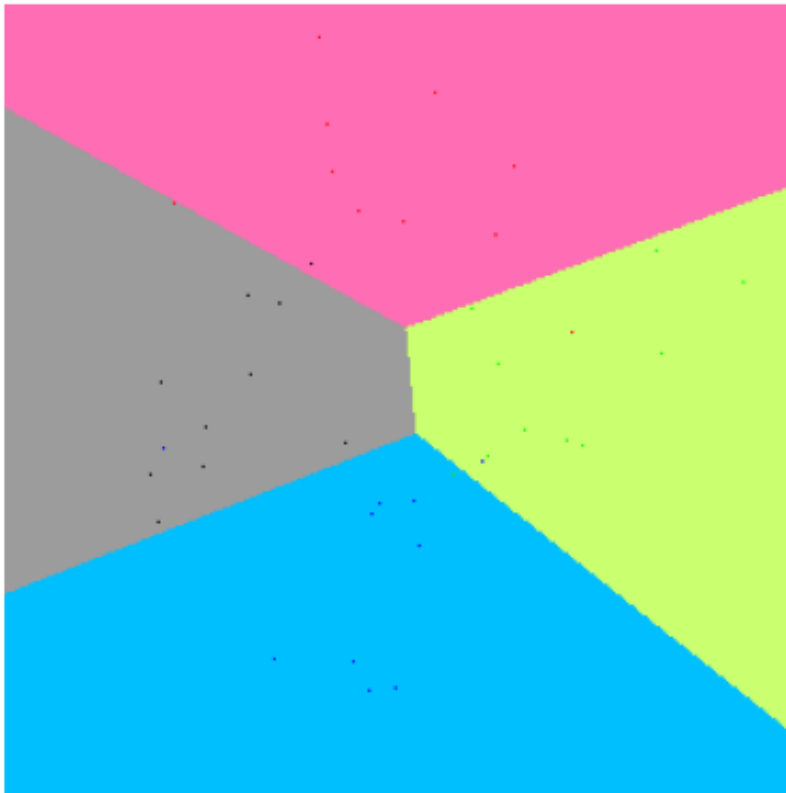
```
iteration 19 loss: 0.3799108091023627 accuracy: 0.8
iteration 39 loss: 0.3592816802660286 accuracy: 0.8
iteration 59 loss: 0.3512540617331573 accuracy: 0.825
iteration 79 loss: 0.34669502157989784 accuracy: 0.825
iteration 99 loss: 0.343720542965191 accuracy: 0.825
iteration 119 loss: 0.34163612060214493 accuracy: 0.825
iteration 139 loss: 0.340109376662506 accuracy: 0.825
iteration 159 loss: 0.33895705208491006 accuracy: 0.825
iteration 179 loss: 0.33806833330879793 accuracy: 0.85
iteration 199 loss: 0.3373716154567217 accuracy: 0.85
iteration 219 loss: 0.33681829827973225 accuracy: 0.85
iteration 239 loss: 0.3363741601377549 accuracy: 0.85
iteration 259 loss: 0.3360144233495191 accuracy: 0.85
iteration 279 loss: 0.3357207569383561 accuracy: 0.85
iteration 299 loss: 0.3354793644084344 accuracy: 0.85
iteration 319 loss: 0.3352797145264293 accuracy: 0.85
iteration 339 loss: 0.3351136719497572 accuracy: 0.85
iteration 359 loss: 0.3349748866300484 accuracy: 0.85
iteration 379 loss: 0.33485835623099924 accuracy: 0.85
```

```
iteration 399 loss: 0.3347601073040041 accuracy: 0.85
iteration 419 loss: 0.334676959738077 accuracy: 0.85
iteration 439 loss: 0.33460635063715927 accuracy: 0.85
iteration 459 loss: 0.33454620123648754 accuracy: 0.85
iteration 479 loss: 0.3344948153856384 accuracy: 0.85
iteration 499 loss: 0.33445080144173667 accuracy: 0.85
```





```
## Visualize Learned Decision Boundary
visualization(X, Y, linear_model)
```



✓ b) Implementation of Multi Layer Perceptron (MLP)

Implement a class `MLP` to add arbitrary layers. You will need to implement the forward function to return results `out` with `x` fed into the model.

```
class MLP(Module):
    """
    Multi Layer Perceptron
```

```

"""
def __init__(self, dimensions):
    """
    Initialize multiple layers here in the list named self.linear_la
    """
    assert isinstance(dimensions, list)
    assert len(dimensions) > 2
    self.linear_layers = []
    for i in range(len(dimensions) - 1):
        self.linear_layers.append(LinearLayer(dimensions[i], dimensi

def __call__(self, x):
    """
    Args:
        x (2d-list): Two dimensional list of Values with shape [batc

    Returns:
        xout (2d-list): Two dimensional list of Values with shape [b
    """
    #TODO Implement this function and return the output of a MLP
    xout = x
    for i, layer in enumerate(self.linear_layers):
        xout = layer(xout)
        if i < len(self.linear_layers) - 1:
            xout = [[val.relu() for val in sample] for sample in xou
    return xout

def parameters(self):
    """
    Get the parameters of each layer

    Args:
        None

    Returns:
        params (list of Values): Parameters of the MLP
    """
    return [p for layer in self.linear_layers for p in layer.paramet

def zero_grad(self):
    """
    Zero out the gradient of each parameter
    """
    for p in self.parameters():
        p.grad = 0

```

Train your MLP model and visualize the decision boundary with ground truth points.

```
## Initialize MLP with Given Parameters
```

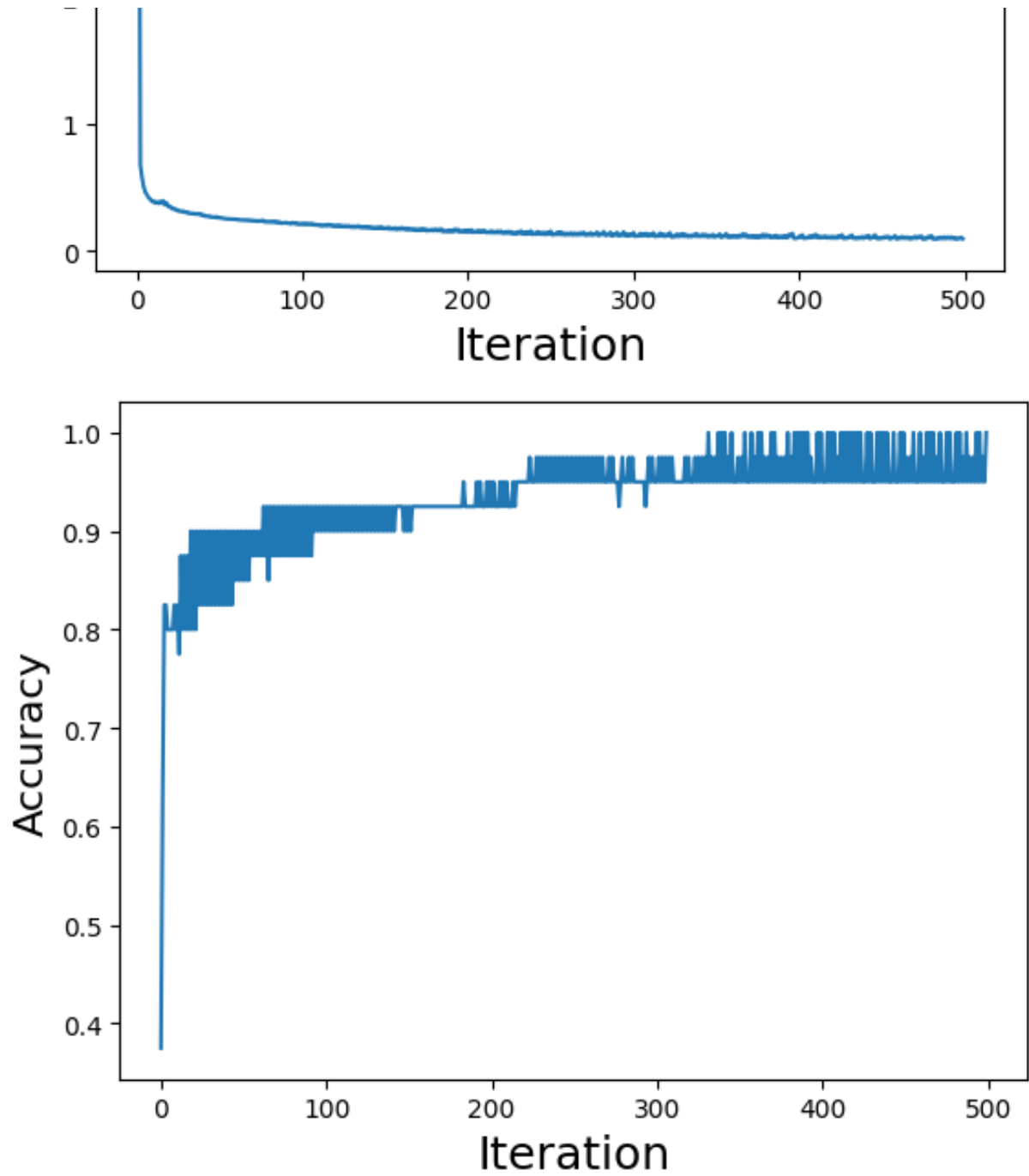
```
mlp_model = MLP([nin, 40, nout])
```

```
## Train the MLP
```

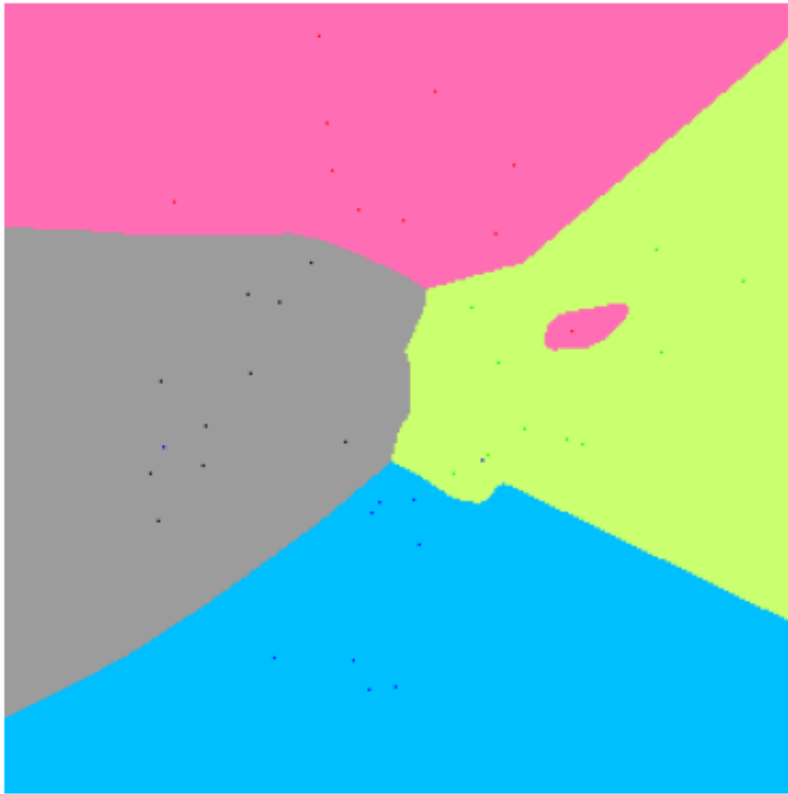
```
train(x, y, mlp_model)
```

```
iteration 19 loss: 0.3504027203544417 accuracy: 0.8
iteration 39 loss: 0.2831405875727273 accuracy: 0.825
iteration 59 loss: 0.24998092287870102 accuracy: 0.875
iteration 79 loss: 0.2296361040627483 accuracy: 0.875
iteration 99 loss: 0.21134667840262533 accuracy: 0.9
iteration 119 loss: 0.19534512089982395 accuracy: 0.9
iteration 139 loss: 0.1834282158275031 accuracy: 0.9
iteration 159 loss: 0.17313088182910708 accuracy: 0.925
iteration 179 loss: 0.1615120676956845 accuracy: 0.925
iteration 199 loss: 0.15089459586686488 accuracy: 0.95
iteration 219 loss: 0.14521187195526286 accuracy: 0.95
iteration 239 loss: 0.1324773273173979 accuracy: 0.975
iteration 259 loss: 0.12724088311923892 accuracy: 0.975
iteration 279 loss: 0.12333174681545366 accuracy: 0.975
iteration 299 loss: 0.12618401265818452 accuracy: 0.95
iteration 319 loss: 0.11543133146013135 accuracy: 0.975
iteration 339 loss: 0.11541117857898764 accuracy: 1.0
iteration 359 loss: 0.11075272938858688 accuracy: 0.975
iteration 379 loss: 0.11047751457229592 accuracy: 1.0
iteration 399 loss: 0.10392639765503497 accuracy: 1.0
iteration 419 loss: 0.1041148459571673 accuracy: 1.0
iteration 439 loss: 0.10120104646336947 accuracy: 1.0
iteration 459 loss: 0.09598793434420078 accuracy: 1.0
iteration 479 loss: 0.10595237573928698 accuracy: 0.95
iteration 499 loss: 0.09873629179255092 accuracy: 1.0
```





```
## Visualize Decision Boundaries  
visualization(X, Y, mlp_model)
```



> Acknowledgement

↳ 1 cell hidden

